

Dzień 5

Helm, monitoring, kubeadm, AI/GPU

Plan dnia

1. **Helm** — package manager + custom charts
2. **Monitoring** — Prometheus + Grafana + Loki
3. **Kubeadm** — klaster od zera (HA: 3 CP + 3 worker)
4. **AI/GPU na K8s** — fake-gpu-operator, MIG, Kueue, DCGM, production practices
5. **Narzędzia developerskie** (cheatsheet)

→ Repo: `day5/`

Helm (D5/02)

Chart = paczka + templates + values

Release = instancja chartu w klastrze

```
helm install my-app ./mychart
helm upgrade --install --atomic my-app ./mychart
helm rollback my-app 1
helm history my-app
helm template ./mychart --debug # render bez install
```

Subkatalogi:

- 01_install_chart/ — instalacja z workshop repo
- 02_happy_panda/ — chart z artifacthub.io (OCI)
- 03_own_chart/ — własny chart (Python+Redis)

→ day5/02_helm/

Helm chart structure

```
mychart/
├── Chart.yaml           # metadata
├── values.yaml          # defaults
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── _helpers.tpl     # reusable snippets
│   └── NOTES.txt        # post-install message
└── charts/              # subcharts
```

--atomic w CI = automatic rollback przy fail

Prometheus stack (D5/04)

```
helm install kube-prometheus-stack prometheus-community/...
```

Zawiera:

- **Prometheus Operator** (CRD: ServiceMonitor, PrometheusRule)
- **Prometheus** (TSDB + scraping)
- **AlertManager** (routing alertów)
- **Grafana** (preinstall'd dashboards)
- **node-exporter** (DaemonSet, per-node metrics)
- **kube-state-metrics** (K8s objects metrics)

→ `day5/04_monitoring_alerting/`

Loki — central logging

```
helm install loki grafana/loki-stack
```

Like Prometheus but for **logs** — LogQL podobny do PromQL.

```
{namespace="default"} |= "error"  
{pod=~"nginx-.*"} | json | response_time > 1000
```

Lekki, kompatybilny z Grafana, **chunks compression** = tani storage.

Alternatywy: ELK (Elasticsearch+Logstash+Kibana), OpenSearch, Splunk, Datadog

Custom Prometheus alerts

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
spec:
  groups:
    - name: app.alerts
      rules:
        - alert: HighErrorRate
          expr: |
            sum(rate(http_requests_total{status=~"5.."}[5m])) > 0.05
          for: 5m
          annotations:
            summary: "Error rate > 5% przez 5 min"
            runbook_url: https://runbooks.example.com/high-error
```

AlertManager → Slack / PagerDuty / Opsgenie / email / webhook

kubeadm: klaster od zera (D5/06)

Cel: 3 CP + 3 worker + kube-vip + Cilium

```
# Na każdym node:  
sudo ./prepare.sh  
  
# Pierwszy CP:  
sudo kubeadm init --config kubeadm-config.yaml --upload-certs  
cilium install --version 1.17.6  
  
# Reszta CP:  
sudo kubeadm join ... --control-plane --certificate-key ...  
  
# Workery:  
sudo kubeadm join ...
```

Alternatywy: kops (AWS), kubespray (Ansible), k3s/RKE2 (lekki), EKS/GKE/AKS (managed)

→ `day5/06_install/`

Sekcja 10 — AI / GPU na K8s

🔥 Najszybciej rosnący segment K8s w 2024-2026.

Wyzwanie szkolenia: nie wymagamy fizycznego GPU.

Rozwiązanie: [fake-gpu-operator](#) (Run:AI) — emuluje cały stos NVIDIA na CPU-only K3d/Kind klastrze.

✅ Scheduling logic, MIG, multi-GPU, taints, autoscaling

❌ Faktyczne CUDA workloady (mock metrics)

→ `day5/07_ai_gpu/`

GPU stack na K8s

```
[Pod: nvidia.com/gpu: 1]
  ↓
[K8s scheduler] ← respektuje extended resources
  ↓
[NVIDIA Device Plugin] ← DaemonSet, gRPC do kubelet
  ↓
[NVIDIA GPU Operator] ← orkiestracja całego stacka:
  ├── Driver DaemonSet
  ├── Device Plugin
  ├── Container Toolkit (mounts CUDA libs)
  ├── DCGM Exporter (metrics → Prometheus)
  ├── MIG Manager
  └── GPU Feature Discovery
```

→ `day5/07_ai_gpu/01_install_fake_gpu/`

MIG — Multi-Instance GPU

A100 (80GB) podzielony hardware-level na "mini-GPU":

Profile	SM	Memory	Max instances
1g.10gb	1/7	10 GB	7
3g.20gb	3/7	20 GB	2
7g.80gb	7/7	80 GB	1 (cały)

```
resources:  
  limits:  
    nvidia.com/mig-1g.5gb: 1
```

Strategia: `single` (cały klaster jeden profile) vs `mixed` (różne)

→ `day5/07_ai_gpu/04_mig_demo/`

Kueue — job queueing dla AI/ML

⚠️ Tylko przegląd (zbyt obszerne na 1 dzień)

`ResourceFlavor` → `ClusterQueue` (quota) → `LocalQueue` → `Workload`

Use case: 3 teamy współdzielą 16 GPU. Team A nie używa? Team B borrowuje. Sprawiedliwe + fair sharing.

Native integracje: Job, JobSet, MPIJob, RayJob, PyTorchJob

Alternatywy: Volcano, YuniKorn

→ `day5/07_ai_gpu/05_kueue_intro.md`

DCGM Exporter + Prometheus

Najważniejsze metryki:

- `DCGM_FI_DEV_GPU_UTIL` — % SM utilization
- `DCGM_FI_DEV_FB_USED` — memory used
- `DCGM_FI_DEV_GPU_TEMP` — thermal (alert > 83°C)
- `DCGM_FI_PROF_PIPE_TENSOR_ACTIVE` — Tensor Cores activity
- `DCGM_FI_DEV_ECC_DBE_V0L_TOTAL` — ECC errors (alert > 0)

Grafana dashboard 12239 — DCGM Exporter Dashboard

→ `day5/07_ai_gpu/06_dcgm_concepts.md`

GPU production practices

- ✓ **Dedicated node pool** (taint `nvidia.com/gpu=true:NoSchedule`)
- ✓ **Spot instances** (70-90% taniej, jeśli checkpointing)
- ✓ **Karpenter** zamiast Cluster Autoscaler (szybszy, multi-instance type, spot fallback)
- ✓ **Image pre-loading** (Spegel) — 5GB pull × 100 nodów = 500GB
- ✓ **Cost monitoring** (OpenCost / Kubecost)

→ `day5/07_ai_gpu/07_production_practices.md`

Sekcja 11 — Narzędzia developerskie (CLI)

(Bez ćwiczeń w repo, tylko cheatsheet)

Tool	Co robi
kubectx	szybsze switch context
kubens	szybsze switch namespace
k9s	terminal UI dla K8s
kube-ps1	shell prompt z context+namespace
stern	multi-pod log tail

Sekcja 11 — Narzędzia developerskie (GUI / dev)

Tool	Co robi
Lens	desktop GUI (multi-cluster)
Headlamp	open-source alternatywa Lens
Telepresence	local dev z proxy do remote cluster
VS Code K8s extension	manifesty, snippets, debug Pod

Co dalej? (continuous learning)

- **Certyfikacje:** CKA, CKAD, CKS (Linux Foundation)
- **CNCF Landscape** — `landscape.cncf.io`
- **TGI Kubernetes** (live show on YouTube)
- **KubeCon** talks (recordings free na YouTube)
- **The New Stack, Daily.dev** dla bieżących newsów

Podsumowanie 5-dniowego kursu

Dzień	Tematy główne
D1	Docker hardening + K8s podstawy
D2	Workloady, AuthN/AuthZ, Gateway API
D3	Scheduling, autoscaling, deployment
D4	Security pełen stack (PSA, Vault, OPA, Falco, Trivy)
D5	Helm, monitoring, kubeadm, AI/GPU

Repo: github.com/<TWOJ_LOGIN>/k8s-training-2026

Dziękuję! 🎉